Post-Quantum Access Tokens for CDNs: Stapling Fixes the Edge, Batching Fixes the Issuer

Dhavan Arjampudi

***Abstract*—CDN signed URLs and cookies rely on one of two kinds of credential. HMAC keys shared with every edge are post-quantum secure, but any compromised edge can use them to mint tokens. RSA, ECDSA and Ed25519 keys would be broken by a quantum computer. We study the design space of post-quantum CDN access tokens against 14 baselines, using native microbenchmarks, a Go edge prototype, real HTTP/2 encoders and a fleet simulator checked against the prototype. Two findings organize the space. *Stapling fixes the edge.* A client sends its per-token ML-DSA-44 signature once per edge connection. The URL then carries a 148-B token, and a full HTTP request costs the edge as much CPU as HMAC (22.33 vs. 22.80 µs). *Batching fixes the issuer.* Pooling every token of an issuer key into one Merkle tree per 10 ms cuts issuer signing CPU 19-fold at $2{,}000$ tokens/s and 144-fold at $20{,}000$ tokens/s. The costs are 268-B tokens and 5.4 ms of extra delay. This scheme, EMBER, is a CDN profile of MTL-mode batch signing. Per-tenant trees, which we recommended before, save only 1.2-fold. Header-compression behavior depends on the client: Chrome adopts a 64 KiB HPACK table, which makes ML-DSA cookies cheap, but curl and Go do not. We derive deployment rules, including when delegated Ed25519 keys suffice.**

***Index Terms*—Access tokens, content delivery networks, HPACK, Merkle trees, ML-DSA, post-quantum cryptography, signed URLs.**

I. Introduction

CONTENT delivery networks (CDNs) protect paywalled video, software downloads and other private objects with *access tokens*. An issuer, usually the content owner's login service, gives an authorized client a short-lived token. The token binds a URL prefix, an expiry time and optional client attributes. The client attaches it to every request: as a query parameter (a *signed URL*), as a cookie, or in a header. Any edge server can check the token before serving from cache, without contacting the origin. A streaming session sends one manifest request and then hundreds of segment requests, so each token is verified hundreds of times, at every edge the client reaches.

Deployed schemes use one of two approaches. Symmetric schemes such as Akamai's EdgeAuth [1] append an HMAC [2] under a key held by every edge. Public-key schemes sign the token: CloudFront key groups accept RSA-2048 or ECDSA P-256 keys [3], and Google Media CDN uses Ed25519 [4]. HMAC-SHA256 is not threatened by quantum computers, but every verifier can also mint tokens. CDNs already hold many customers' secrets [5], [6], so a single compromised edge exposes every path the key covers. The public-key schemes keep edges secret-free, but Shor's algorithm [7] would recover their signing keys from long-lived public keys. Authentication has no harvest-now-decrypt-later exposure. The reason to migrate now is lead time: token formats, SDKs and edge verifiers must change before a cryptographically relevant quantum computer (CRQC) exists, and industry roadmaps target post-quantum authentication by about 2029 [8].

The obvious replacement is one FIPS 204 ML-DSA signature per token [9], [10]. It produces a 3,350-B token, which every segment request then repeats. Merkle batch signing [11]–[13] is the standard remedy for large signatures, and our earlier version of this work applied it to CDN tokens as EMBER. Review of that version, together with new measurements, showed two things. At the operating point we had recommended (per-tenant trees, 10 ms epochs), most trees hold a single token, so batching saves almost nothing. And most of the benefit we had attributed to the tree came from *detaching* the signature and sending it once per edge. This paper therefore separates the two mechanisms and measures each against strong alternatives.

Contributions.

1) **A trust and threat comparison** of 14 token schemes (Section III, Table VII). It covers who can mint tokens after an edge compromise, which post-quantum assumption each scheme rests on, and token size. We state plainly that HMAC is already post-quantum secure.
2) **Stapling fixes the edge** (Section VI-B). Stapled per-token ML-DSA (M2) sends the 2,420-B signature once per edge connection. It matches HMAC in whole-request edge CPU in a Go prototype, and it is the smallest post-quantum option on the wire in URL form, and within 0.4% of the smallest in cookie form.
3) **Batching fixes the issuer** (Section VI-C). EMBER is a CDN profile of MTL-mode and batch signatures. It is not a new primitive, and its root stapling is MTL's full signature sent once. EMBER pays off only when one tree is pooled per issuer key. We map the operating region (tokens per tree against rate and epoch length) where it pays and where it does not.
4) **HPACK behavior of real clients** (Section VI-D). Chrome 153 adopts an edge-advertised 64 KiB HPACK table, which makes even 3.3 KB ML-DSA cookies cost 25.7 B per request. curl/nghttp2 and Go clamp the table to 4096 B.
5) **A prototype and a simulator**, with the simulator checked against the prototype on miss counts (24 of 24 metric rows, 12 configurations) and with the cases where it fails reported (Section VI-G).
6) **Deployment guidance** (Section VII). Use M2 below

D. Arjampudi is with Andhra University, Visakhapatnam, India (e-mail: *[editable]*).

[Editable: funding statement, ORCID, corresponding-author note.]

about 10^3 tokens/s per issuer key and pooled EMBER with stapling above it. Use delegated Ed25519 only if a key cannot be broken within its exposure window.

II. Background and Related Work

A. CDN Access Tokens and Header Compression

A CDN token consists of *claims* and an *authenticator*. Typical claims are a URL prefix, an expiry time and optionally a client IP range. Media CDN appends `Signature=` and an unpadded base64url Ed25519 signature [4]. EdgeAuth appends `hmac=` [1]. Size limits depend on where the token is carried. RFC 9110 recommends that URIs of at least 8000 octets be supported [14]. Some legacy clients accepted only about 2 KB, for example Internet Explorer's limit of 2,083 characters [15]. RFC 6265 requires at least 4096 B per cookie [16]. Per-token ML-DSA-44 tokens fit the RFC limits but not the legacy one.

In HTTP/2, request headers are compressed with HPACK [17]. A repeated header field, such as a cookie, is sent as a short index once it sits in the dynamic table. Two parties set the size of that table. The decoder, which for requests is the edge, advertises a maximum with SETTINGS_HEADER_TABLE_SIZE [18]. The encoder, which is the client, may use any smaller size and signals its choice with a Dynamic Table Size Update [17]. Whether a 3.3 KB cookie stays indexed therefore depends on the client as well as the CDN (Section VI-D).

B. Post-Quantum Signatures and Their Cost

FIPS 204 specifies ML-DSA, whose ML-DSA-44/65/87 signatures are 2420, 3309 and 4627 B [9]. FIPS 205 specifies SLH-DSA, with 7856 B for 128s and 17,088 B for 128f [19]. RFC 9964 registers ML-DSA for JOSE and COSE and cautions that ML-DSA may not suit settings that need small signatures [10]. Measurements of post-quantum TLS find that signature *size*, more than CPU, drives the cost [20]–[23]. CDNs already use ML-KEM key exchange at the edge and toward origins [24], [25], and at least one accepts ML-DSA certificates for origin authentication [26]. KEMTLS removes handshake signatures [27], [28], but tokens are verified offline, outside any handshake. We combine hybrid signatures with the AND-combiner analyzed by Bindel et al. [29].

C. Merkle Batch Signing and Signed Tree Heads

Merkle's tree authentication [11] underlies all of the following schemes. Aguilar-Melchor et al. formalize *batch signatures*, in which one signature covers a tree of messages, and they evaluate them for TLS [12]; a TLS batch-signing draft [30] follows the same idea. *MTL mode* [13], [31] signs ladders over a message series. Each message is then authenticated by a *condensed* signature, a path to a ladder rung. A verifier that lacks the rung needs the *full* signature, which includes the signed ladder. The DNSSEC profiles ML-DSA-MTL [32] and SLH-DSA-MTL [33] apply this pattern. ML-DSA-MTL uses the same ML-DSA-44 root and condensed-path structure as EMBER. Merkle Tree Certificates (MTC) [34], [35] batch certificate issuance and ship selected signed tree heads, called landmarks, to relying parties ahead of time, and a CDN operator is building a CA on them [36]. MTAC profiles MTC for AI-agent identity, not runtime authorization [37]. SLH-DSA's tweakable-hash framework [38] supplies our node domain separation.

EMBER's token is an MTL condensed signature over a per-epoch series, and its root stapling is MTL's full signature sent once per edge. Its pull and push modes correspond to MTC landmark distribution. We searched the literature and IETF drafts up to November 2026 and found no work that applies these mechanisms to CDN signed URLs or cookies. What this paper adds is therefore not a mechanism. It is a measurement of where each mechanism pays off in a CDN, including where it does not.

D. Other Token Designs

Delegated credentials [39] let a long-term key certify short-lived keys, and our baseline B11 uses this pattern. Macaroons [40] attenuate HMAC bearer tokens but keep HMAC's shared-secret trust. DPoP [41] binds a token to a client key, a concern orthogonal to how the token is authenticated. Post-quantum Privacy Pass [42] provides unlinkable tokens of 85–175 KB, for a different goal. Sharing keys across servers is known to enlarge the damage of a single compromise [6], [43].

III. System and Threat Model

Entities. An *issuer* I holds a signing key sk with key id kid. The issuer is the tenant's login service or an issuer that the CDN operates for its tenants. *Edges* hold pk, a cache of verified tokens or roots, and a revocation set. In the batched design, a *root service* publishes signed roots as immutable, cacheable objects behind the CDN's shield tier. *Clients* obtain a token and then send a manifest request and K segment requests, normally to one edge.

Why not simply HMAC? HMAC-SHA256 tokens are post-quantum secure. Their weakness is that every verifier can also mint tokens. One compromised edge, whether through memory disclosure, an insider or partner hardware, yields a key that mints tokens for every edge and every path the key covers. A content owner using several CDNs must give each vendor the minting key. And no issuer accountability exists. Public-key tokens give edges only pk. The benefit is limited: a compromised edge can always serve whatever it caches without checking tokens. Asymmetric tokens therefore remove only *lateral* minting, that is, tokens accepted by other edges, other CDNs or other tenants. Table VII gives this blast radius for each scheme.

Adversary. The adversary $\mathcal{A}$ (i) controls the client–edge network and can replay tokens it observes; (ii) may compromise any subset of edges and shields, including their caches; (iii) can obtain tokens for claims of its choice as a legitimate client, while salts and nonces stay issuer-chosen; and (iv) is quantum. It can run Shor's algorithm against any public key and generic quantum search against hash functions. $\mathcal{A}$ does not obtain sk. An issuer compromise is handled by rotating kid. Theft and replay of bearer tokens are unchanged from the status quo and

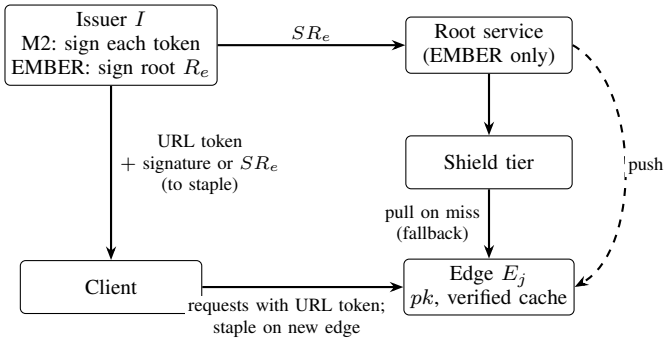


Fig. 1. Stapled post-quantum tokens. In M2 the client staples its token's ML-DSA signature on the first request to each edge. In EMBER it staples the signed epoch root SR_e (MTL's "full" signature) and the URL carries a Merkle path. Pull from the root service through the shield tier and push are EMBER fallbacks.

are mitigated as they are today, by IP or ASN claims, short validity, or DPoP-style binding [41].

Goals. **G1**: an honest edge accepts no claims that I did not issue or that I revoked. **G2**: edges hold no minting secret. **G3**: validity is bounded; for EMBER, the expiry is at most $T_{\text{start}}(e) + \Delta_e + V_{\max}$ plus a skew tolerance τ. **G4**: an authentication path reveals nothing about other clients' claims. **G5**: an edge that cannot verify a token fails closed. The operational requirements are small tokens, per-request edge cost close to HMAC, bounded issuance delay and issuer cost, revocation, and dissemination traffic that is negligible next to the content served.

IV. Design Space

All schemes authenticate the same canonical claim string, so only the authenticator and its delivery differ. Fig. 1 shows the two post-quantum designs this paper focuses on.

A. Per-Token Signatures, Cached or Stapled

B5/B6 put an ML-DSA-44/65 signature in every token. **B5v** adds an edge cache of verified tokens, keyed by SHA-256 of the token, so verification runs once per (token, edge). The 3.3 KB token is still sent and hashed on every request. **M2** (stapled per-token ML-DSA-44) separates the signature from the token. The URL token is the claims followed by `:S=`b64url($kid \,\|\,$ 16-B nonce). On its first request to an edge, the client sends the 2,420-B signature over (claims, nonce) in a header. The edge verifies it, caches $H(\mathit{ref} \,\|\, \text{claims})$ as verified until the expiry, and from then on hashes about 138 B per request. M2 is the $N = 1$ special case of EMBER, and it is also B5v without the per-request resend. Its security is the EUF-CMA security of ML-DSA plus the collision resistance of the cache key.

B. EMBER: Pooled Epoch-Merkle Batching

Notation. H is SHA-256 truncated to $n \in \{16, 32\}$ bytes. The tweakable hash is $\mathsf{Th}(T, x) = \text{trunc}_n(H(T\|x))$ with a fixed-length 26-B tweak $T =$ `"EMBERv1\0"`$\|\mathit{kid}\|e\|\mathit{type}\|\ell\|j\|\mathit{tenant}$, so every node position of every epoch is a distinct function [38]. Leaf i of epoch e is $L_i = \mathsf{Th}(T_{\text{LEAF}}(e, i), r_i\|\text{enc}(c_i))$, where r_i is a 128-bit salt that the issuer draws *after* it receives the claims c_i. Inner nodes are $\mathsf{Th}(T_{\text{NODE}}(e, \ell, j), \text{left}\|\text{right})$, and the tree has the RFC 9162 shape [44], with depth $d = \lceil \log_2 N_e \rceil$. The signed epoch root is $SR_e = (\mathit{kid}, e, T_{\text{start}}, \Delta_e, N_e, R_e, \sigma_e)$, where $\sigma_e = \mathsf{Sig}(sk, \texttt{"EMBER-root"}\|\mathit{kid}\|e\|T_{\text{start}}\|\Delta_e\|N_e\|R_e)$ is ML-DSA-44 with the FIPS 204 context string `EMBER-root`. In the hybrid variant, σ_e is an Ed25519 signature and an ML-DSA-44 signature over the same message, and both must verify.

Token. The URL token is `URLPrefix=...:Expires=...:Tenant=...:E=`*blob*, where the blob is the base64url encoding of version, algorithm, kid, e, i, r_i and the path, $30 + nd$ raw bytes in all. With c_{enc} bytes of encoded claims,

$$S_{\text{tok}} = c_{\text{enc}} + 3 + \lceil 4(30 + nd)/3 \rceil. \tag{1}$$

Our encoder implements Eq. (1). With a 60-character path ($c_{\text{enc}} = 118$) it yields 161, 268, 332 and 460 B for $N = 1$, 2^5, 2^8 and 2^{14}, against 148 B for M2.

Issuance and pooling. The issuer collects requests for Δ seconds, or until a count cap $N_{\max}$ is reached, and then closes the epoch. Closing builds the tree ($2N - 1$ hashes), signs the root once, and returns to each client its token and SR_e (Algorithm 2, Appendix A). The unit of batching is the *pool*, meaning the set of tenants that share a tree. Batching saves signatures only when a pool receives many tokens per Δ. With P pools the issuer makes at most P/Δ signatures per second, independent of the token rate. Pooling lets one issuer key mint tokens for every tenant in the pool. It is therefore appropriate only when the tenants already delegate issuance to the CDN or belong to one trust domain. Tenant is part of the claims and of the leaf tweak, so leaves cannot be replayed across tenants.

Verification is given in Algorithm 1. Revocation uses short $V_{\max}$ (1 h by default), signed lists of revoked (e, i) pairs at 8 B each, or withdrawal of a whole epoch root.

C. Root Dissemination

An edge cannot verify an EMBER token until it holds R_e. There are four ways to get it there. **Pull**: the edge fetches SR_e through the shield on a miss, with coalesced fetches. **Push**: the issuer fans each root out to all edges, at fleet bandwidth $BW_{\text{push}} = T_{\text{active}}\, E\, (|\sigma|+40)/\Delta$ for T_{active} active pools. **Period roll-up**: every m epochs one signed bundle lists the m roots. **Stapling**: the client sends SR_e in a header, either on its first request (*staple-first*) or on every request to an edge it has not used before (*staple-all*). The stapled header is the 2,461-B signed root, 3,298 B as an HTTP/1.1 header line. With staple-all, verification does not depend on the root service at all, which matters because edges fail closed: during an outage of the root service or the shield tier, pull-only edges reject every token of an epoch they have not cached, while stapled tokens still verify. For Poisson token arrivals in pull mode, the

Algorithm 1 Edge verification of an EMBER token

procedure VERIFY(tok, url, now)
  parse claims c and $(kid, e, i, r, \text{path})$; reject if malformed or kid unknown
  reject if $now > c.Exp + \tau$ or $c.Exp > T_{\text{start}}(e) + \Delta_e + V_{\max} + \tau$
  reject if URL prefix or IP binding fails, (e, i) revoked, or $i \geq N_e$
  $R \leftarrow RootCache[kid, e]$
  if $R = \perp$ **then**
    if a stapled SR_e is present **then** verify σ_e (per-client rate limit); cache
    else fetch SR_e via shield (coalesced, negative-cached); verify; cache
    end if
    on failure or timeout: **return** reject ▷ fail closed
  end if
  $x \leftarrow \mathsf{Th}(T_{\text{LEAF}}(e, i), r \| \text{enc}(c))$
  for each sibling s_ℓ in path **do** $x \leftarrow \mathsf{Th}(T_{\text{NODE}}(e, \ell, \cdot), \text{order}(x, s_\ell))$
  end for
  return $x \stackrel{\text{ct}}{=} R$
end procedure

expected number of root misses per token at edges with traffic shares w_v is

$$M(\Delta) = \frac{1}{\lambda\Delta} \sum_{v=1}^{E} \left(1 - e^{-\lambda\Delta w_v}\right). \tag{2}$$

The added delay per token is $D = \Delta/2 + t_{\text{epoch}} + M(\Delta)F_{\text{miss}}$, where F_{miss} is the root round trip plus t_v (pull), or t_v alone (stapling). When $\lambda\Delta \ll E$, $M \to 1$: almost every token is the first at its edge in its epoch, so only stapling or push avoids the round trip.

D. Security

Theorem 1. *Suppose an adversary $\mathcal{A}$ makes q_h queries to Th and the issuer evaluates Th at $p \leq \sum_e (2N_e - 1)$ tree positions. Then there exist $\mathcal{B}_1, \mathcal{B}_2$ with about the same running time such that $\text{Adv}^{\text{uf-tok}}_{\text{EMBER}}(\mathcal{A}) \leq \text{Adv}^{\text{euf-cma}}_{\mathsf{Sig}}(\mathcal{B}_1) + \text{Adv}^{\text{sm-tcr}}_{\mathsf{Th},p}(\mathcal{B}_2)$.*

Proof sketch: Let R^* be the root the edge used. If the issuer never produced R^* for (kid, e^*), then the verified σ is a forgery, which $\mathcal{B}_1$ outputs. Otherwise $R^* = R_{e^*}$ is honest, and since $i^* < N_{e^*}$ is enforced, an honest leaf i^* exists whose claims differ from c^*. The honest and adversarial hash chains start from different inputs and reach the same root. So at some position they have different inputs and equal outputs under the same tweak. The honest input there contains the salt r_{i^*}, which was chosen after $\mathcal{A}$ fixed c_{i^*}, or a value derived from it. $\mathcal{A}$ therefore hits a fixed honest target, which is an sm-tcr break; distinct tweaks rule out multi-target speed-ups [38]. Replays across epochs, tenants or kid change the tweak and fall into the second case. ∎

This is a sketch, not a new proof. For $n = 32$, plain collision resistance suffices, matching batch signatures [12]. The root and period-bundle signatures use distinct context strings. The hybrid root is an AND-combiner, at least as strong as its stronger component [29]. sm-tcr is defined for a keyed hash family. Our tweak contains kid and the epoch but no random public seed (SLH-DSA's PK.seed). We therefore recommend adding a 16-B random seed per kid to the tweak, carried in SR_e. This adds 16 B to the staple and nothing to tokens. We did not measure this variant. *Truncation:* ML-DSA-44 is a category-2 parameter set [9]. With $n = 16$, the generic second-preimage bound is about 2^{128} classically and 2^{64} with Grover. The *token* therefore offers category-1 security, the same generic level as SLH-DSA-128's 16-B hashes [19]. Use $n = 32$ for higher categories (+341 B at $N = 2^{16}$). The issuer-chosen salt is mandatory, because $\mathcal{A}$ must not control both inputs of a collision.

Other threats. Downgrade: once a tenant has switched, edges must reject classical tokens for that tenant. Privacy: N_e in a public root leaks issuance volume, and the pair (e, i) identifies a token, as a per-token signature already does. Padding the tree to 2^d hides the volume. It slowed our Python issuer by 12.45% at $N = 1000$, but the worst case is $N = 2^k + 1$, where the analytic native cost rises by 38.9% at $N = 2^{14} + 1$. DoS: tokens naming fake epochs trigger root fetches, and fake staples trigger signature verifications (Section VI-H). Replaying a *valid* staple costs nothing at an edge that has the root cached, and costs at most one verification per (kid, e) elsewhere. Per-client limits bound both cases.

TABLE I
SCHEMES COMPARED. ALL ARE IMPLEMENTED IN PYTHON; † MARKS SCHEMES THAT ARE ALSO IN THE GO PROTOTYPE.

ID	Scheme
B1†	HMAC-SHA256, key shared with all edges (EdgeAuth-like)
B2†, B3†, B4	Ed25519 (Media CDN); ECDSA P-256 and RSA-2048 (CloudFront)
B5†, B6†, B7, B8	per-token ML-DSA-44/65, SLH-DSA-128s/128f
B9†	per-token hybrid Ed25519 + ML-DSA-44
B10	multi-MAC: one 16-B HMAC tag per verifier domain, $k \in \{1, 3, 10\}$
B11†	delegated Ed25519 key, certified per lifetime L by ML-DSA-44
B12†	per-epoch HMAC key wrapped with ML-KEM-768 [45] to each edge
B13	MTL mode, one series per key [31]
M2†	**stapled per-token ML-DSA-44** (Section IV)
B5v†, B5c	B5 with an edge verify cache; B5 split over two cookies
O1†/O2/O3/O4†	EMBER, $n = 16$/$n = 32$/SLH-DSA-128s root/hybrid root

V. EVALUATION METHODOLOGY

Schemes. Table I lists the 14 baselines (B1–B13 and M2) and the variants. Macaroons and DPoP are discussed only qualitatively (Section II). In B13 the MTL series grows over $V_{\max}$. An MTL series restarted at every epoch is structurally the same as EMBER, so we do not list it separately.

Testbed and implementations. All experiments ran on one KVM guest with 4 vCPUs (Xeon at 2.1 GHz, AVX-512, SHA-NI) and 15 GB of RAM, running Linux 6.18. Primitive costs come from a native C benchmark against OpenSSL 3.5.4 (EVP interface, 5 fresh processes pinned to one core). We report the median of the run medians together with the [min–

max] of the run medians. The Go prototype uses Go 1.24.7 and cloudflare/circl 1.6.1, and Python (pqcrypto 1.0.0) is a secondary source. We ran the native suite twice, separated by a VM reboot. Absolute values differ by 15–20% (ML-DSA-44 verification took 100.9 vs. 122.3 µs), while ratios are stable. All tables and all simulations use the second run, except the epoch-controller study and the simulator consistency check (Appendix B). Rerunning the parameter sweep with second-run costs left misses unchanged bit for bit and moved D by at most 0.1 ms, so the controller conclusions, whose effects are all far below 10 ms, cannot change.

Prototype. The Go prototype consists of an edge (`net/http`; root cache with coalescing and a negative cache; optional verified-token cache; V virtual edges), an issuer and root service, and an open-loop load generator. Client, edge and origin run in separate network namespaces joined by userspace link emulators (`netem` is unavailable): one-way delays of 10 ms and 37.5 ms, 1 Gbit/s, MTU 1500. *P1* measures edge CPU cost. It is a closed loop over the edge namespace's loopback with the edge on one core, 64 keep-alive connections, $K = 150$ requests per token and 5 runs. Its primary metric is edge CPU time per HTTP request. P1 measures CPU, not the network. *P3* measures root dissemination over the emulated links with $V \in \{10, 50\}$ virtual edges, $\Delta \in \{0.1, 1, 10\}$ s and $\lambda \in \{20, 200\}$/s, in pull and staple-first modes: 12 configurations, each with a 60 s window and 5 runs. *P2* measures latency under load (Appendix B). TLS is not included.

Wire bytes and real clients. Request-header bytes per session (1 manifest and 150 segment requests) are encoded with Go's HPACK at 4096-B and 64 KiB tables. To observe real encoders, a raw-frame HTTP/2 server advertises a table size and logs each request's header-block length while a client fetches 150 objects with a 3,311-B cookie value. The clients are headless Chrome 153 (3 runs), curl 8.5.0/nghttp2 1.59.0 and the Go x/net client. We did not test Firefox or Safari.

Fleet simulator. The simulator is an epoch-level Monte Carlo engine driven by the native costs. Its default fleet has $E = 500$ edges with Zipf(0.8) traffic shares and 50 shields, and $T = 1000$ tenants with Zipf(1.0) rates and aggregate rate $\bar{\lambda} \in \{200, 2000, 20{,}000\}$/s. Sessions have K geometric with mean 150, 4 s apart, and each segment is re-routed to another edge with probability $p_{\text{move}} = 0.0005$ (about 7% of sessions move once). RTTs are lognormal: edge–shield 15 ms, shield–root 60 ms, token receipt to first request 120 ms. All of these are assumptions; the sensitive ones are swept (Appendix B). The pooling policies are per-tenant trees, 8 shards, 10 or 50 regional pools, and one pool per issuer key. Each is run with a fixed Δ, work-conserving closing (WC, close whenever the signer is idle) or M2. CIs are Student-t 95% intervals over 5 seeds with common random numbers. Unless stated, their half-widths are below 1% of the mean.

Hypotheses. We fixed hypotheses H1–H9 in the design document before running any experiment (repository commit `3148c65`, 2026-11-05 18:07 UTC). Appendix C reports every outcome.

TABLE II
NATIVE PRIMITIVE COSTS (OPENSSL 3.5.4, SECOND RUN): MEDIAN OF 5 RUN MEDIANS [MIN–MAX OF RUN MEDIANS], µS. KEY SIZES ARE RAW KEYS (ECDSA: 65-B POINT; RSA: 270-B MODULUS AND EXPONENT), NOT SPKI.

Scheme	pk	sig	sign	verify
HMAC (200 B)	–	32	0.30 [0.30–0.30]	0.30 [0.30–0.30]
Ed25519	32	64	30.1 [30.1–30.4]	94.6 [93.8–96.2]
ECDSA P-256	65	71	23.0 [22.9–31.1]	69.6 [69.5–70.0]
ML-DSA-44	1312	2420	408.8 [386.9–452.4]	100.9 [100.2–120.4]
ML-DSA-65	1952	3309	676.5 [673.5–682.7]	154.8 [154.6–165.8]
SLH-DSA-128s	32	7856	306,464	288.1 [282.4–320.0]
Path d=0 / d=16	–	–	–	0.24 / 2.34

TABLE III
WHOLE-REQUEST EDGE COST IN THE GO PROTOTYPE (P1; ONE EDGE CORE, SAME SESSION FOR ALL SCHEMES, $K = 150$; MEAN [95% t-CI] OVER 5 RUNS).

Scheme	token (B)	req/s	edge CPU (µs/req)
B1 HMAC	100	43,392	22.80 [22.23, 23.36]
M2 stapled	83	44,254	**22.33** [21.60, 23.05]
O1 EMBER (2^{10}, warm)	309	37,786	26.16 [25.51, 26.82]
B5v + cache	3,285	23,232	42.53 [41.89, 43.18]
B5 ML-DSA-44	3,284	13,079	75.58 [72.12, 79.04]

VI. RESULTS

A. Primitive Costs and Naive Post-Quantum Tokens

Table II lists the native costs. ML-DSA-44 verification takes 100.9 µs, about the same as Ed25519 (94.6 µs). An EMBER path of depth $d = 16$ takes 2.34 µs and $d = 0$ takes 0.24 µs. HMAC over 200 B takes 0.30 µs. A path at $d = 16$ is thus 43× cheaper than OpenSSL's ML-DSA-44 verification but 7.9× HMAC. In Go, a whole EMBER token verification takes 3.07 µs, against 32.76 µs for circl's ML-DSA-44, a ratio of 10.7×. circl verifies 3.6–4.0× faster than OpenSSL's portable code.

The token sizes below are measured with a 60-character path. Per-token signatures are large: 3,350 B for B5, 4,535 B for B6 and 10,598 B for B7. B5 fits RFC 9110's 8000-octet URL and RFC 6265's 4096-B cookie but not a 2 KB legacy limit. B6 exceeds the cookie minimum. The classical and symmetric baselines are small: 166 B (B1), 209 B (B2), 214 B (B11) and 171 B (B12). Splitting B5 over two cookies (B5c) gives at most 1,687 B per cookie.

B. Stapling Fixes the Edge

Table III compares whole HTTP requests, including Go's request parsing, on one edge core. With stapling, per-token ML-DSA-44 (M2) costs 22.33 µs of edge CPU per request, which is indistinguishable from HMAC (22.80 µs). In one run, the edge's own counter recorded 7,419 staple verifications in 1,108,422 requests. EMBER with a warm 2^{10} tree costs 26.16 µs, 17% more than M2, because it hashes a path on every request. B5v still parses and hashes its 3.3 KB token on every request and costs 42.53 µs, and plain B5 costs 75.58 µs. An earlier quiet campaign covering 12 schemes (Appendix B) puts per-token Ed25519, ECDSA and hybrid tokens above per-token ML-DSA-44 in Go. Its absolute values differ by about

TABLE IV
OPERATING POINTS WITH EVERY METRIC AT THE SAME CONFIGURATION (SIMULATED; STAPLE-ALL; TOKEN SIZE FROM THE ENCODER). ISSUER: CORES OF ML-DSA-44 SIGNING AND TREE WORK. EDGE: CRYPTO µS PER REQUEST. HOT: ML-DSA VERIFICATIONS/S AT THE BUSIEST EDGE. D: MEAN ADDED DELAY.

Configuration	N_e=1 (%)	tok/tree (mean)	token (B)	issuer (cores)	edge (µs)	hot (/s)	D (ms)
$\bar{\lambda} = 2000$ tokens/s							
M2 per-token	100	1	148	0.818	0.95	165.8	0.41
per-tenant, 10 ms	71.5	1.22	161	0.674	1.01	162.4	5.41
per-tenant, WC	99.8	1.00	161	0.818	0.95	165.8	0.41
8 shards, 10 ms	8.8	2.79	204	0.295	1.20	150.1	5.41
1 pool, WC	63.6	1.26	161	0.650	1.01	162.6	0.54
1 pool, 10 ms	0	20.0	268	**0.0425**	1.51	88.1	5.43
1 pool, 100 ms	0	200	332	0.0057	1.71	15.2	50.6
$\bar{\lambda} = 20{,}000$ tokens/s							
M2 per-token	100	1	148	8.18	0.95	1,658	0.41
per-tenant, 10 ms	41.0	1.85	183	4.43	1.17	1,420	5.41
1 pool, WC	0.03	8.31	247	1.000	1.38	1,244	0.62
1 pool, 10 ms	0	200	332	**0.057**	1.71	151	5.57
$\bar{\lambda} = 200$ tokens/s							
M2 per-token	100	1	148	0.082	0.95	16.6	0.41
1 pool, 10 ms	13.6	2.31	204	0.036	1.16	15.4	5.41

10% for B1 (20.54 vs. 22.80 µs), which we attribute to host variability between runs.

Stapling also removes most of the byte cost. M2's URL token is 148 B in the encoder (claims plus a 16-B nonce reference), against 3,350 B for B5. M2 needs no batching wait, so its mean added delay with staple-all is 0.41 ms in simulation (Table IV). For EMBER the edge comparison is unfavorable. M2 pays one 100.9 µs verification per (token, edge), amortized over K requests. EMBER pays one root verification per (root, edge) plus, on every request, $\mathrm{path}(d) - \mathrm{path}(0)$. EMBER is cheaper at the edge only if $K < 100.9/(\mathrm{path}(d) - \mathrm{path}(0))$, which gives $K < 154$ for $d = 5$ and $K < 96$ for $d = 8$. Our workload has a mean K of 150. The simulator agrees: per-request edge crypto is 0.953 µs for M2 against 1.509 µs for pooled EMBER at $\bar{\lambda} = 2000$/s. Both are microsecond-level.

C. Batching Fixes the Issuer, Only When Pooled

Fig. 2 and Table IV answer where batching pays. Take the per-tenant, 10 ms configuration that our earlier version recommended, at $\bar{\lambda} = 2000$/s. There, 71.5% of tokens sit in single-token trees and trees hold 1.22 tokens on average. Even in the busiest tenant decile, which carries 69% of the tokens, 59.4% of tokens are alone in their tree. The issuer makes 1,646 root signatures/s for 2,000 tokens/s: 0.674 cores against 0.818 for per-token signing, a 1.2× saving. Work-conserving closing makes almost every tree a single token. Pooling all tenants of an issuer key into one tree per 10 ms changes this. Each tree then holds 20 tokens on average (200 at 20,000/s), and the issuer makes at most $1/\Delta = 100$ signatures/s regardless of the rate. Issuer CPU falls to 0.0425 cores at $\bar{\lambda} = 2000$/s, 19× less than M2, and to 0.057 cores at 20,000/s, 144× less. Root verifications at the busiest edge fall from 1,658/s to 151/s. The costs are a larger URL token (268 or 332 B instead of 148 B), $\Delta/2$ of issuance wait (5.4 ms mean), and 1.6–1.8× the fleet edge crypto CPU (0.456 vs. 0.288 cores at $\bar{\lambda} = 2000$/s; 5.18 vs. 2.88 at 20,000/s). Regional pools reduce verifications at the hottest edges (89.5/s with 50 regions) but help the issuer little, because each region's rate is small. At $\bar{\lambda} = 200$/s, pooling saves 0.046 cores, which does not justify the extra delay and bytes.

Microbenchmark capability. The figures we previously advertised hold only at large N. A native issuer core produces 41,774 tokens/s at $N = 16$, 469,254 at $N = 2^8$ and 1,275,990 at $N = 2^{14}$, against 2,446 for per-token ML-DSA-44 signing. A token is 460 B at $N = 2^{14}$. Reaching $N \geq 2^{14}$ requires $\lambda\Delta \geq 1.6 \times 10^4$, which no simulated configuration with $\Delta \leq 0.1$ s attains.

TABLE V
REQUEST-HEADER BYTES PER SESSION (1 + 150 REQUESTS ON ONE CONNECTION; GO HPACK ENCODER AT 4096-B AND 64 KIB TABLES; SAME PLACEMENT FOR ALL SCHEMES).

	URL, HTTP/2		Cookie, HTTP/2	
Scheme	4096 B	64 KiB	4096 B	64 KiB
B1 HMAC	26,706	24,918	9,148	9,121
B11 deleg. Ed25519	33,277	30,825	9,308	9,161
B5 ML-DSA-44	457,831	410,359	458,568	11,667
B5c (2 cookies)	–	–	460,833	11,932
M2 + staple	27,048	25,273	12,114	11,711
EMBER 2^5 + staple	43,401	39,966	13,165	11,853
EMBER 2^8 + staple	52,389	47,958	13,466	11,894
EMBER 2^{14} + staple	69,161	63,012	14,090	12,008

D. Bytes on the Wire and Real HPACK Encoders

Table V compares schemes in the same placement. In URL form, stapled EMBER at the pooled tree size 2^5 uses 0.095× B5's bytes under HTTP/2 with a 4096-B table, but M2 uses less still: 27,048 B, within 1.3% of HMAC. In cookie form, the HPACK table size decides the outcome. With a 64 KiB table, even unstapled B5 (11,667 B) is smaller than stapled EMBER (11,853 B). Every post-quantum option then costs 11.7–12.0 KB per session, close to HMAC's 9.1 KB. The stapled header costs EMBER 2,968 B per HTTP/2 session (3,298 B as an HTTP/1.1 header line) and M2 2,921 B.

Which table size applies depends on the client's encoder (Section II). When our server advertised 64 KiB, Chrome 153 adopted it, emitting a table-size update to 65,536 in all 3 runs. The 3.3 KB cookie then cost 25.7 B per request after the first, and 150 requests cost about 6.5 KB of header blocks in total. With a 4096-B table, Chrome re-sent the cookie each time, at 2,993–3,000 B per request. curl/nghttp2 and the Go client both clamped the table to 4096 B whatever the server advertised, and the Go source confirms a 4096-B default encoder limit. A CDN that advertises a large table thus makes ML-DSA cookies affordable for Chrome users but not for other clients. Stapled designs do not depend on this setting.

E. Dissemination and the Stapling Assumption

Table VI compares the dissemination modes with per-tenant and pooled trees side by side. Pull makes nearly every first request wait for a root (1.069 misses per token), because $\lambda\Delta \ll E$ (Eq. (2)). Per-tenant push costs $5.0 \cdot 10^{-3}$ of served

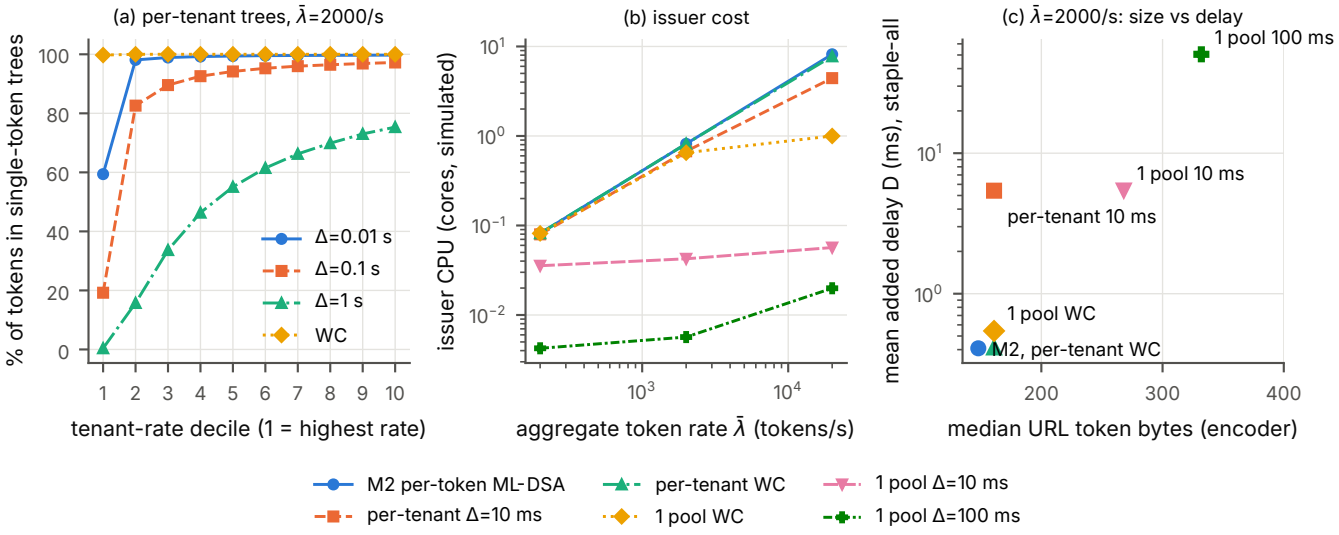


Fig. 2. Operating region (simulated, second-run costs, 5 seeds). (a) Share of tokens in single-token trees by tenant-rate decile, per-tenant trees. (b) Issuer CPU against aggregate rate. (c) Median URL token size (encoder) against mean added delay with staple-all.

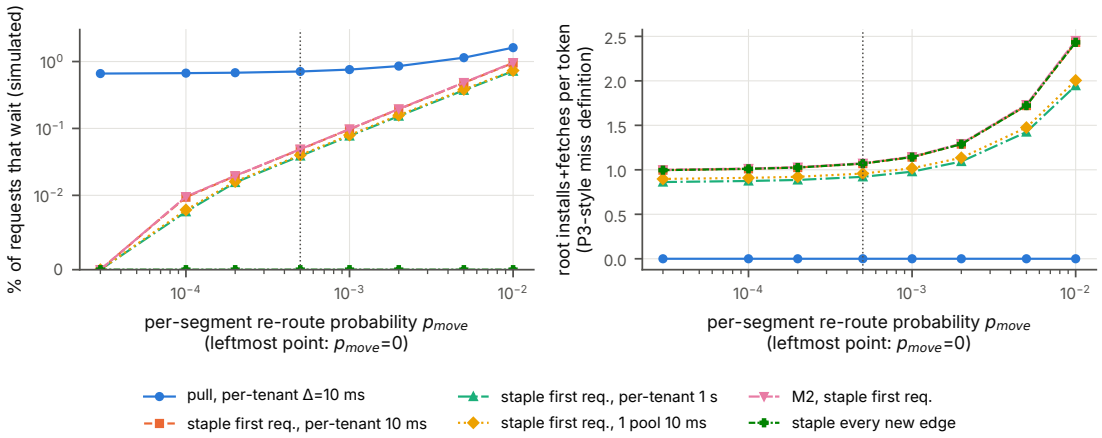


Fig. 3. Sensitivity of stapling to mid-session re-routes (simulated, $\bar{\lambda} = 2000$/s). Left: share of requests that wait for a root or signature. Right: root installs plus fetches per token (the prototype's miss definition). The dotted line marks the default $p_{\text{move}} = 0.0005$.

TABLE VI
DISSEMINATION MODES, EMBER AT $\bar{\lambda} = 2000$/S, $\Delta = 10$ MS (SIMULATED, 5 SEEDS). T: PER-TENANT TREES. P8: 8 POOLED SHARDS. WAIT: SHARE OF REQUESTS THAT WAIT FOR A ROOT.

	Per-tenant trees (T)				8 shards (P8)	
Mode	miss/tok	D (ms)	wait %	root/served	miss/tok	D (ms)
pull	1.069	96.1	0.710	$2.4 \cdot 10^{-5}$	1.054	94.3
staple-first	0.073	11.6	0.049	$1.6 \cdot 10^{-6}$	0.071	11.5
push	0.622	19.7	0.412	$5.0 \cdot 10^{-3}$	0.616	19.6
pull+roll-up	1.045	93.2	0.694	$4.5 \cdot 10^{-5}$	1.003	89.8
push+roll-up	1.012	91.3	0.672	$1.0 \cdot 10^{-4}$	0.997	89.7

bytes and 2.27 GB/s of fleet root traffic, and it still misses 0.622 times per token, because the 150 ms push delay exceeds the 120 ms between token receipt and first request. Roll-up removes only 1.4–2.3% of misses: from 6,098 to 6,011 per 10^6 requests at $\Delta = 1$ s, and from 1.069 to 1.045 per token at 10 ms. Roots arrive before any period closes, so roll-up is useful only for carrying revocation lists and bounding root state. Root traffic is at most $1.0 \cdot 10^{-4}$ of served bytes in every mode except per-tenant push.

The staple-first figure of 0.049% waiting requests is a *model output that rests on one assumption*. Its misses come entirely from the assumed mid-session re-route probability p_{move} (Fig. 3). The share is 0 at $p_{\text{move}} = 0$, 0.049% at the default $5 \cdot 10^{-4}$, 0.48% at $5 \cdot 10^{-3}$ and 0.95% at 10^{-2}, and M2 behaves the same. *Staple-all* removes waiting entirely at every p_{move}, at a cost of 2,501 to 6,254 B of stapled roots per session over that range. We recommend staple-all. Our prototype did not exercise re-routes.

F. When Delegated Ed25519 (B11) Suffices

B11 is the strongest classical competitor. Its token is 214 B, its issuer signs in 30.1 µs per token, and its certificate traffic is

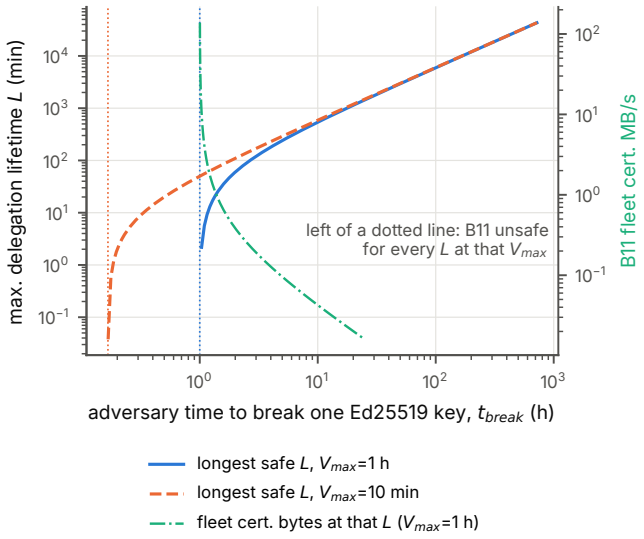


Fig. 4. B11 safety against quantum key recovery (analytic). A delegated key is exposed for its lifetime L plus $V_{\max}$. It is safe only if one Ed25519 key takes longer than $L + V_{\max}$ to break. Right axis: fleet certificate traffic at that L ($E = 500$, $T = 1000$).

0.383 MB/s at $L = 1$ h, compared with 8.41 MB/s for EMBER pull. A delegated Ed25519 public key is exposed from the publication of its certificate until the last token signed under it expires, a window of $L + V_{\max}$. B11 is therefore quantum-safe only if an adversary needs more than $L + V_{\max}$ to break one key (Fig. 4). Shortening L is cheap: at $L = 60$ s the cost is 16.7 certificate fetches per second per edge, 0.17% of an edge core and 23 MB/s of fleet traffic. The binding constraint is therefore $V_{\max}$. If a key can be broken in less than $V_{\max}$ (1 h in our workload), no L makes B11 safe and M2 or EMBER is needed. One published estimate for a 256-bit elliptic-curve key reports one key every 38 h on a strictly 2D-local architecture, and lower cost with non-local connections [46]. Under such estimates B11 with a short L remains safe for some time. On issuer cost, B11 (30.1 µs/token) beats M2 (408.8) and per-tenant EMBER (337.2), but loses to pooled EMBER (21.2 at 2,000/s, 2.83 at 20,000/s).

G. Simulator Validation Against the Prototype

We ran the simulator with the prototype's P3 configuration and measured RTT distribution (Fig. 5). P3 covers 12 configurations, each scored on two metrics, giving 24 rows. In P3 a stapled-root install counts as a miss, so pull and staple-first show the same miss counts; the comparison uses the same definition on both sides. Misses per token and root bytes per token pass in 24 of 24 rows (relative error at most 15% or the simulator inside the prototype's CI), with a maximum error of 4.4%. Mean miss latency passes for pull at $\Delta \leq 1$ s (76.9–78.5 ms measured against 75.9 ms simulated) and fails elsewhere. At $\Delta = 10$ s the prototype measures 118–137 ms. There, timer-aligned epochs release about 4,400 tokens whose first requests queue at the one-core edge, and the simulator models no queueing. Stapled installs take 0.074–0.086 ms against a modelled 0.026 ms, because the model omits decoding of the 3.3 KB header. The validation covers neither re-routes nor the shield tier. We therefore claim validation only for miss and byte counts at small scale ($V \leq 50$).

H. Other Results

DoS (one simulated edge, 10^4 fake tokens/s, second-run costs). Fake epochs spread over all tenants cause 9,270–9,636 root fetches/s, 4,630 verifications/s and 0.484 of a core; coalescing and negative caching do not help, because keys rarely repeat. Restricted to one low-rate tenant, coalescing cuts fetches from 7,680/s to 65/s. A flood of fake staples costs 1.026 cores. A limit of one staple per second per client (100 clients) reduces this to 0.0269 cores. Per-client limits are therefore required for both staples and fetches. *Hybrid root.* At $\Delta = 1$ s in pull mode, verifying the Ed25519 half of a hybrid root raises simulated edge CPU from 1.405 to 1.982 µs per request (+41.1%). In the simulator a root is verified roughly once per session, about once per 141 requests with per-tenant 10 ms trees, so the extra 94.6 µs Ed25519 verification is amortized over few requests. In the warm P1 configuration the root was verified once per run, and O4 and O1 differ by 0.09% per request. *End-to-end.* The mean added latency per request is 0.0503 ms for EMBER staple-first, 0.0049 ms for B5v, 0.101 ms for B5 and 0.486 ms for EMBER pull (p99.9 112 ms). B5v therefore beats stapled EMBER by 10× on the mean. All schemes are below 1 ms at p99.

VII. Discussion

A. Deployment Guidance

Table VII summarizes the trust and cost trade-offs. Our recommendations follow from the measurements.

1. If edges may hold a minting key, use HMAC. HMAC (B1) is post-quantum and the cheapest option. Per-domain multi-MAC (B10) confines a multi-CDN breach to one vendor at HMAC cost.

2. Otherwise, staple. Detaching the signature from the token and sending it once per edge connection fixes the edge for any signature scheme. M2 matches HMAC in edge CPU (Table III) and is the smallest post-quantum option in URL form; in cookie form it is within 0.4% of the smallest (Table V). Clients should staple on every new edge contact, not only on the first request (Fig. 3). Stapling also removes the root service as an availability dependency.

3. Batch only when the issuer is the bottleneck, and then pool. Per-token signing costs the issuer $\lambda \times 408.8$ µs, about 0.41 cores per 10^3 tokens/s per key. Pooled EMBER at $\Delta = 10$ ms costs at most $1/\Delta$ root signatures per second plus hashing. In our runs this stayed at or below 0.057 cores up to 20,000 tokens/s, and at $\lambda \geq 10^3$/s trees hold at least about 10 tokens. Below about 10^3 tokens/s per issuer key, the saving is under half a core: 0.046 cores at 200/s. That does not justify 5.4 ms of extra delay, 120–184 more bytes per token and the extra path hashing at the edge, so M2 is the better choice. Above that rate, use one pooled tree per issuer key with $\Delta \approx 10$ ms and staple-all. Pooling requires that the tenants of a pool trust a shared issuer key. With per-tenant keys, batching saves only

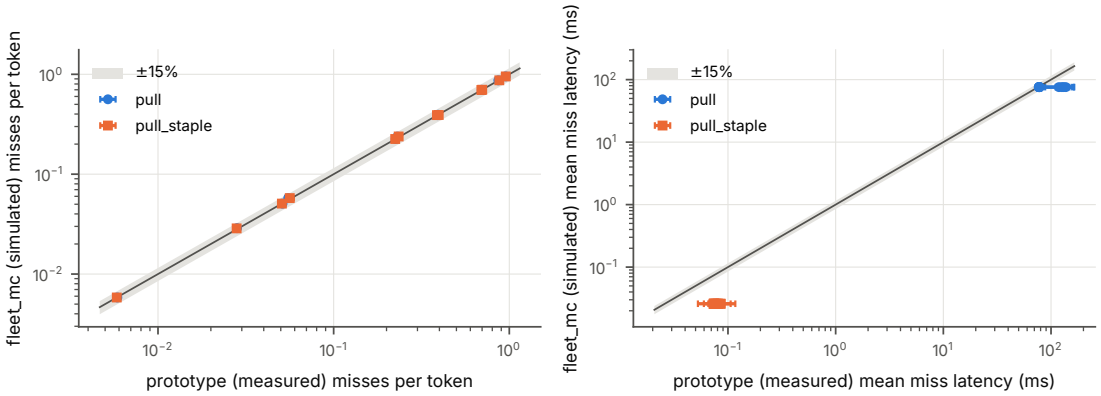


Fig. 5. Simulator (50 seeds, fed the measured RTTs) against the prototype (P3, 5 runs, 95% CI) for 12 configurations in pull and staple-first modes. Left: misses per token. Right: mean miss latency.

TABLE VII
TRUST AND COST COMPARISON. BYTES: ENCODER URL TOKEN, 60-CHARACTER PATH. VERIFY: NATIVE OPENSSL 3.5.4 MEDIANS. DISSEMINATION: SIMULATED FLEET TRAFFIC FOR $E = 500$ ($\bar{\lambda} = 2000$/S) OR PER-SESSION STAPLE BYTES.

Scheme	Secret at edge	One compromised edge mints for	PQ assumption	Bytes	Verify (µs)	Dissemination
B1 HMAC	yes (key lifetime)	all edges, all paths of the key	unconditional	166	0.30	none
B10 multi-MAC, k=3	yes	its own domain	unconditional	187	0.30	none
B12 KEM-wrapped HMAC	yes (epoch keys)	all edges, live epochs	unconditional	171	0.30	283.5 MB/s key push
B2 Ed25519	no	none	none (Shor)	209	94.6	none
B11 delegated Ed25519	no	none	no break within $L+V_{\max}$	214	94.6 per (key, edge)	0.383 MB/s ($L = 1$ h)
B5 ML-DSA-44	no	none	unconditional	3,350	100.9 per request	none
B5v + verify cache	no	none	unconditional	3,350	100.9 first, 2.19 hit	none
M2 stapled ML-DSA-44	no	none	unconditional	148	100.9 per (token, edge), $\leq$0.21 hit	staple 2,921 B/session (h2)
EMBER pooled, staple-all	no	none	unconditional	268	0.89 path (d=5) + 100.9 per (root, edge)	staple 2,968 B/session (h2)
EMBER per-tenant, pull	no	none	unconditional	161–247	0.24–0.75 path (d=0–4) + root	8.41 MB/s ($\Delta = 1$ s)

1.2×, and M2 should be used. Work-conserving closing self-limits issuer CPU to one core per pool at sub-millisecond delay (0.62 ms at 20,000/s), which suits pools whose rate fits on one signer core.

4. Consider B11 if its assumption holds. Delegated Ed25519 is smaller and cheaper to disseminate. It is appropriate when breaking one Ed25519 key is believed to take longer than $L+V_{\max}$ (Section VI-F). Short $V_{\max}$ is the effective lever.

5. Operational settings. Use a node size of $n = 16$ for category-1 tokens and $n = 32$ above that. Add a per-*kid* public seed to the tweak. Set $V_{\max} = 1$ h and a skew tolerance $\tau = 5$ s. Apply per-client limits on staples and fetches, and fetch roots only for configured *kid*s and recent epochs. Do not deploy per-tenant push or rely on roll-up to reduce misses.

B. Limitations

- *One VM.* All prototype components share one 4-vCPU KVM guest. Network namespaces and a userspace link emulator stand in for `netem`, and there is no physical NIC and no TLS. P1 runs over loopback, so it measures edge CPU per request, not network effects. Separate P1 campaigns differed by about 10% in absolute value.
- *Two native runs.* Absolute costs differ by 15–20% across a VM reboot. Everything except the controller study and the simulator consistency check uses the second run (Appendix B).
- *Simulator scope.* The simulator is validated against the prototype only for miss and byte counts, at small scale ($V \leq 50$), and without re-routes or a shield tier. It has no edge-CPU or link queueing, so it under-predicts pull-miss latency for timer-aligned bursts (118–137 ms measured vs. 75.9 ms). Workload parameters (Zipf exponents, RTTs, K, p_{move}) are assumptions, and p_{move} alone determines staple-first waiting.
- *Clients.* We tested Chrome 153, curl/nghttp2 and Go, but not Firefox or Safari.
- *Security.* We give a proof sketch, not a machine-checked proof. The public-seed variant was not implemented. Bearer-token replay is unchanged from the status quo. Pooling concentrates minting in one key.

VIII. CONCLUSION

Post-quantum CDN tokens face two separate bottlenecks, and each has its own fix. At the edge, the problem is sending and verifying a 2.4 KB signature on every request. Stapling the signature once per edge connection solves it: in our prototype, per-token ML-DSA-44 then costs the edge as much as HMAC, with a 148-B URL token. At the issuer, the problem is one

ML-DSA signature per token. Pooled epoch-Merkle batching (EMBER, a CDN profile of MTL-mode batch signing) solves it, cutting issuer CPU 19× at 2,000 tokens/s and 144× at 20,000 tokens/s. Batching pays only with pooled trees, and only above roughly 10^3 tokens/s per issuer key. Per-tenant trees, which we previously recommended, save just 1.2×. Header compression can hide large cookies, but only for clients that adopt a large HPACK table, such as Chrome. Open questions are measurements in a multi-PoP deployment, re-routes in the prototype, a simulator with queueing, and Firefox and Safari encoder behavior.

Data Availability

All code, raw measurements, aggregated CSV files and figure scripts are in the author's repository. `NUMBERS_TRACE.md` maps every number in this paper to its source file and row. [Editable: insert public archive DOI or URL before submission.]

Declaration on the Use of AI Tools

[Editable: AI-based tools were used to assist with code, experiments and drafting. The author reviewed and takes responsibility for all content. Adapt to the venue's policy.]

Acknowledgment

[Editable.]

Appendix A
EMBER Issuance

Algorithm 2 Issuer: epoch closing and publication

procedure ISSUE(c)
  $r \leftarrow \$\{0,1\}^{128}$; append (c, r, future) to the pool's queue Q; **return** await future
end procedure
procedure CLOSEEPOCH ▷ timer Δ, or $|Q| = N_{\max}$, or signer idle (WC)
  $B \leftarrow Q$; $Q \leftarrow [\,]$; $e \leftarrow e+1$; $N \leftarrow |B|$ ▷ rotate kid before e or i wraps
  $L_i \leftarrow \mathsf{Th}(T_{\text{LEAF}}(e,i),\ B[i].r \,\|\, \mathrm{enc}(B[i].c))$ for $i < N$
  $(R, \mathit{levels}) \leftarrow$ BUILDTREE(e, L)
  $\sigma \leftarrow \mathsf{Sig}(sk, \texttt{"EMBER-root"} \| \mathit{kid} \| e \| T_{\text{start}} \| \Delta \| N \| R)$
  publish SR_e at `/ember/`kid`/`e (immutable); push if configured
  for $i < N$ **do**
    resolve $B[i].\mathit{future}$ with $(c_i, \mathit{kid}, e, i, r_i, \text{PATH}(\mathit{levels}, i))$ and SR_e
  end for
end procedure

Appendix B
Secondary Results

Epoch controller (first-run costs). We also built an adaptive controller. It estimates the token rate with an exponentially weighted moving average and, at each epoch close, chooses Δ from a 25-point logarithmic grid in $[0.01, 60]$ s to minimize D, subject to size, signer-capacity and 2 s issuance-latency constraints. Before the experiment we fixed a demotion rule: if the controller beat the best fixed Δ, chosen in hindsight, by less than 10 ms or 10%, it would be reported as a guideline. Over a 24-h diurnal workload in pull mode with per-tenant trees, the controller chose the grid floor of 10 ms almost all day. Its paired difference to the best fixed Δ was -0.001 ms [-0.004, 0.002] at $\bar{\lambda} = 200$/s and $+0.005$ ms [0.001, 0.010] at 2000/s (10 seeds each), and -0.83 ms [-0.91, -0.75] at 20,000/s (5 seeds, 1-h peak). The rule therefore triggered. Work-conserving closing beat the controller by 4.93 and 4.41 ms at 200 and 2000/s, and fixed $\Delta \geq 3$ s violated the SLO in every 5-min bin. These runs used first-run costs. The second-run sweep moved D by at most 0.1 ms (96.17→96.08 ms for pull at 10 ms), so the conclusion stands. With stapling, F_{miss} is nearly zero and the optimum is simply the smallest Δ the issuer can sustain.

Simulator consistency (first-run costs). An independent event-driven simulator agrees with the Monte Carlo engine to within 0.56% on misses per token, 0.09% on D and 0.08% on issuance wait, over 45 metric rows ($\bar{\lambda} = 200$/s, $E = 100$, 1 h, 5 seeds). This checks the implementation for internal consistency only. It is not a validation.

Latency under load (P2). We ran P2 open loop over the emulated 21 ms link. Its load levels were set from an earlier, contended capacity campaign. Measured against the quiet P1 capacities, the nominal 50% and 80% levels correspond to 0.35–0.40 and 0.57–0.64 utilization. At about 0.6 utilization, the per-run median added p50 relative to HMAC, with [min, max] over runs, was -2.60 ms [-6.29, 0.55] for O1, +12.14 ms [3.17, 16.52] for B5 and +11.75 ms [2.61, 203.82] for B5v, one of whose runs collapsed. We attribute the B5 and B5v tails to the userspace link emulator, which carried 3.3 KB URLs at roughly 260–450 Mbit/s against about 624 Mbit/s in a bulk test. We did not verify this.

Earlier capacity campaign. A quiet P1 campaign over 12 schemes, with edge utilization ≥ 0.99 in all 60 runs, measured edge CPU per request of 20.54 µs for B1, 22.91 for O1, 22.93 for O4 (hybrid root), 37.38 for B5v, 66.24 for B5, 71.04 for Ed25519, 98.04 for ECDSA P-256 and 115.74 for the per-token hybrid.

Sensitivity. An SLH-DSA-128s root puts its 306 ms signature on the issuance path and raises D from 0.575 to 0.881 s (pull, $\Delta = 1$ s). Halving the RTTs leaves staple-first D at 0.503 s. The simulator's parameter defaults and sweep ranges are listed in Section V and in the released `sim_params.json`.

Appendix C
Hypothesis Outcomes

References

[1] Akamai Technologies, "EdgeAuth-Token-Java: Akamai edge authorization token," GitHub repository, hMAC-SHA256 token generation for cookie, query string and header tokens. Accessed 2026-11-05. [Online]. Available: https://github.com/akamai/edgeauth-token-java

[2] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, Feb. 1997. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2104

TABLE VIII
HYPOTHESES FIXED BEFORE THE EXPERIMENTS (COMMIT `3148c65`) AND THEIR OUTCOMES.

Hypothesis	Outcome	Evidence
H1 EMBER token <512 B to $N{=}2^{20}$; $\leq$ 0.25× B5c bytes/session	partly	503 B at 2^{16}, 588 B at 2^{20}; 0.144× (h2 URL), 0.256× (HTTP/1.1); M2 smaller still
H2 verify $\leq 5\times$ HMAC and $\geq 10\times$ cheaper than ML-DSA	partly	native 7.9× / 43×; Go 3.6× / 10.7×; whole request 1.15× HMAC
H3 issuer $\geq 100\times$ per-token (Python), $N \geq 2^8$	cond.	holds at $N \geq 2^8$, reached only with pooled trees
H4 root bytes $< 0.1\%$; roll-up $\geq 5\times$ fewer misses	partly	$\leq 1.04{\cdot}10^{-4}$ except per-tenant push ($5.0 \cdot 10^{-3}$); roll-up 1.4–2.3%
H5 adaptive beats best fixed Δ	refuted	$\leq$0.83 ms; demoted
H6 $< 1\%$ requests wait; p99 edge CPU < 1 ms	yes (model)	0.049% at default p_{move}, 0 with staple-all; p99 $\leq$2.61 µs
H7 hybrid root $< 1\%$ edge cost	mixed	+0.09% in P1; +41.1% simulated
H8 simulator within 15% of prototype	partly	misses and bytes 24/24 rows; latency 8/24
H9 native issuer $\geq 50\times$ per-token; B11/B12 comparable	partly	192–522× at $N{=}2^8$–2^{14}; B11 issues faster than per-tenant EMBER

[3] Amazon Web Services, "Specify signers that can create signed URLs and signed cookies (Amazon CloudFront developer guide)," Product documentation, key groups accept RSA-2048 or ECDSA-256 public keys. Accessed 2026-11-05. [Online]. Available: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-trusted-signers.html

[4] Google Cloud, "Media CDN: Generate signatures," Product documentation, ed25519-signed URLs, URL prefixes and cookies. Accessed 2026-11-05. [Online]. Available: https://cloud.google.com/media-cdn/docs/generate-signatures

[5] J. Liang, J. Jiang, H. Duan, K. Li, T. Wan, and J. Wu, "When HTTPS meets CDN: A case of authentication in delegated service," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2014, pp. 67–82.

[6] F. Cangialosi, T. Chung, D. Choffnes, D. Levin, B. M. Maggs, A. Mislove, and C. Wilson, "Measurement and analysis of private key sharing in the HTTPS ecosystem," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS '16)*. ACM, 2016, pp. 628–640.

[7] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.

[8] B. Westerbaan, "Cloudflare targets 2029 for full post-quantum security," Cloudflare Blog, Apr. 2026. [Online]. Available: https://blog.cloudflare.com/post-quantum-roadmap

[9] National Institute of Standards and Technology, "Module-lattice-based digital signature standard," NIST, FIPS 204, Aug. 2024.

[10] M. Prorock and O. Steele, "ML-DSA for JSON object signing and encryption (JOSE) and CBOR object signing and encryption (COSE)," RFC 9964, May 2026. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9964

[11] R. C. Merkle, "A certified digital signature," in *Advances in Cryptology – CRYPTO '89*, ser. LNCS, vol. 435. Springer, 1990, pp. 218–238.

[12] C. Aguilar-Melchor, M. R. Albrecht, T. Bailleux, N. Bindel, J. Howe, A. Hülsing, D. Joseph, and M. Manzano, "Batch signatures, revisited," in *Topics in Cryptology – CT-RSA 2024*, ser. LNCS, vol. 14643. Springer, 2024, pp. 163–186.

[13] A. Fregly, J. Harvey, B. S. Kaliski Jr., and S. Sheth, "Merkle tree ladder mode: Reducing the size impact of NIST PQC signature algorithms in practice," in *Topics in Cryptology – CT-RSA 2023*, ser. LNCS, vol. 13871. Springer, 2023, pp. 415–441.

[14] R. T. Fielding, M. Nottingham, and J. Reschke, "HTTP semantics," RFC 9110, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110

[15] Microsoft, "Maximum URL length is 2,083 characters in Internet Explorer (KB 208427)," Microsoft Support, accessed 2026-11-06. [Online]. Available: https://support.microsoft.com/kb/208427

[16] A. Barth, "HTTP state management mechanism," RFC 6265, Apr. 2011. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6265

[17] R. Peon and H. Ruellan, "HPACK: Header compression for HTTP/2," RFC 7541, May 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7541

[18] M. Thomson and C. Benfield, "HTTP/2," RFC 9113, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9113

[19] National Institute of Standards and Technology, "Stateless hash-based digital signature standard," NIST, FIPS 205, Aug. 2024.

[20] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-quantum authentication in TLS 1.3: A performance study," in *Network and Distributed System Security Symposium (NDSS)*, 2020.

[21] C. Paquin, D. Stebila, and G. Tamvada, "Benchmarking post-quantum cryptography in TLS," in *Post-Quantum Cryptography (PQCrypto 2020)*, ser. LNCS, vol. 12100. Springer, 2020, pp. 72–91.

[22] M. Sosnowski, F. Wiedner, E. Hauser, L. Steger, D. Schoinianakis, S. Gallenmüller, and G. Carle, "The performance of post-quantum TLS 1.3," in *Companion of the 19th International Conference on emerging Networking EXperiments and Technologies (CoNEXT 2023)*. ACM, 2023, pp. 19–27.

[23] P. Kampanakis and W. Childs-Klein, "The impact of data-heavy, post-quantum TLS 1.3 on the time-to-last-byte of real-world connections," in *NDSS Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb)*, 2024, iACR ePrint 2024/176. [Online]. Available: https://eprint.iacr.org/2024/176

[24] Cloudflare, "Cloudflare now uses post-quantum cryptography to talk to your origin server," Cloudflare Blog, Sep. 2023. [Online]. Available: https://blog.cloudflare.com/post-quantum-to-origins/

[25] Akamai, "Future-proofing the Internet: Akamai achieves end-to-end PQC," Akamai Blog, Aug. 2026. [Online]. Available: https://www.akamai.com/blog/security/2026/aug/future-proofing-internet-akamai-achieves-end-to-end-pqc

[26] Cloudflare, "Post-quantum authentication to origins is now supported," Cloudflare Blog, 2026. [Online]. Available: https://blog.cloudflare.com/post-quantum-authentication-to-origins/

[27] P. Schwabe, D. Stebila, and T. Wiggers, "Post-quantum TLS without handshake signatures," in *ACM SIGSAC Conference on Computer and Communications Security (CCS '20)*. ACM, 2020, iACR ePrint 2020/534.

[28] S. Celi, A. Faz-Hernández, N. Sullivan, G. Tamvada, L. Valenta, T. Wiggers, B. Westerbaan, and C. A. Wood, "Implementing and measuring KEMTLS," in *Progress in Cryptology – LATINCRYPT 2021*, ser. LNCS, vol. 12912. Springer, 2021, pp. 88–107.

[29] N. Bindel, U. Herath, M. McKague, and D. Stebila, "Transitioning to a quantum-resistant public key infrastructure," in *Post-Quantum Cryptography (PQCrypto 2017)*, ser. LNCS, vol. 10346. Springer, 2017, pp. 384–405.

[30] D. Benjamin, "Batch signing for TLS," IETF Internet-Draft draft-ietf-tls-batch-signing-00 (expired), Jan. 2020. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-ietf-tls-batch-signing

[31] J. Harvey *et al.*, "Merkle tree ladder (MTL) mode signatures," IETF Internet-Draft draft-harvey-cfrg-mtl-mode-09, work in progress, Mar. 2026. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-harvey-cfrg-mtl-mode-09

[32] A. Kaizer, J. Harvey, B. S. Kaliski Jr., and S. Sheth, "ML-DSA-MTL for DNSSEC," IETF Internet-Draft draft-kaizer-dnsop-ml-dsa-mtl-dnssec-01, work in progress, Aug. 2026. [Online]. Available: https://www.ietf.org/archive/id/draft-kaizer-dnsop-ml-dsa-mtl-dnssec-01.html

[33] A. Fregly *et al.*, "SLH-DSA-MTL for DNSSEC," IETF Internet-Draft draft-fregly-dnsop-slh-dsa-mtl-dnssec-06, work in progress, Mar. 2026. [Online]. Available: https://datatracker.ietf.org/doc/draft-fregly-dnsop-slh-dsa-mtl-dnssec/

[34] D. Benjamin, D. O'Brien, B. Westerbaan, L. Valenta, and F. Valsorda, "Merkle tree certificates," IETF Internet-Draft draft-ietf-plants-merkle-tree-certs-05, work in progress, Jul. 2026. [Online]. Available: https://www.ietf.org/archive/id/draft-ietf-plants-merkle-tree-certs-05.txt

[35] L. Valenta, C. Patton, V. Gonçalves, and B. Westerbaan, "Keeping the Internet fast and secure: introducing Merkle tree certificates," Cloudflare Blog, Oct. 2025. [Online]. Available: https://blog.cloudflare.com/bootstrap-mtc/

[36] Cloudflare, "Building a post-quantum certificate authority with Merkle tree certificates," Cloudflare Blog, 2026. [Online]. Available: https://blog.cloudflare.com/pq-ca-with-mtcs/

[37] R. Burls, "Merkle tree agent certificates (MTAC)," IETF Internet-Draft draft-burls-mtac-01, work in progress, Oct. 2026. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-burls-mtac-01

[38] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, "The SPHINCS$^+$ signature framework," in *Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS '19)*. ACM, 2019, pp. 2129–2146.
[39] R. Barnes, S. Iyengar, N. Sullivan, and E. Rescorla, "Delegated credentials for TLS and DTLS," RFC 9345, Jul. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9345
[40] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud," in *Network and Distributed System Security Symposium (NDSS)*, 2014.
[41] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. B. Jones, and D. Waite, "OAuth 2.0 demonstrating proof of possession (DPoP)," RFC 9449, Sep. 2023. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9449
[42] G.-V. Policharla, B. Westerbaan, A. Faz-Hernández, and C. A. Wood, "Post-quantum privacy pass via post-quantum anonymous credentials," IACR Cryptology ePrint Archive, Paper 2023/414, 2023. [Online]. Available: https://eprint.iacr.org/2023/414
[43] D. Springall, Z. Durumeric, and J. A. Halderman, "Measuring the security harm of TLS crypto shortcuts," in *Proceedings of the 2016 Internet Measurement Conference (IMC '16)*. ACM, 2016, pp. 33–47.
[44] B. Laurie, E. Messeri, and R. Stradling, "Certificate transparency version 2.0," RFC 9162, Dec. 2021. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9162
[45] National Institute of Standards and Technology, "Module-lattice-based key-encapsulation mechanism standard," NIST, FIPS 203, Aug. 2024.
[46] D. Litinski, "How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates," arXiv:2306.08585, 2023. [Online]. Available: https://arxiv.org/abs/2306.08585